Introduction to Hash Cryptography

Assignment 5 | Subject: Cryptography and Network Security (BCA309T)

Q1. Define hash cryptography. Explain the fundamental principles and techniques of cryptographic hash functions with suitable examples.

1.1 Definition

Hash cryptography is the branch of cryptography that uses **cryptographic hash functions** to convert data of any size (a message, file or password) into a fixed-size string of bits called the **hash value**, **message digest** or **fingerprint**. A hash function is **one-way**: it is easy to compute the digest from the input but computationally infeasible to recover the input from the digest. Unlike encryption, hashing uses no key and is not meant to be reversed.

Mathematically, a hash function is written as **h = H(M)**, where M is a message of arbitrary length and h is a digest of fixed length (for example 256 bits for SHA-256).

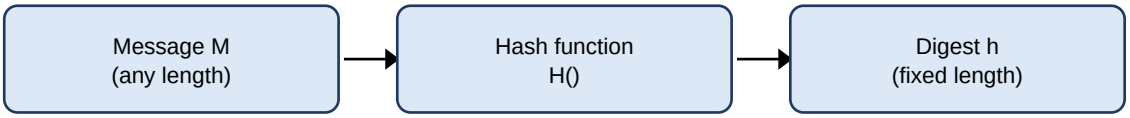

Figure 1: Basic working of a hash function

1.2 Fundamental Principles

- **Deterministic:** the same input always produces the same digest.
- **Fixed-length output:** whether the input is one character or a 5 GB file, the digest length is constant.
- **Efficiency:** the digest can be computed quickly for any input.
- **Pre-image resistance (one-way):** given h, it is infeasible to find M such that H(M) = h.
- **Avalanche effect:** a tiny change in the input (even one bit) changes about half the bits of the output.
- **Collision resistance:** it is infeasible to find two different inputs with the same digest.

1.3 Example: Deterministic output and avalanche effect

The SHA-256 digests of two almost identical messages are completely different:

```
SHA-256("hello")
= 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

SHA-256("Hello")
= 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Only the case of one letter changed, yet the two digests share no visible similarity. This is the avalanche effect.

1.4 Techniques Used to Build Cryptographic Hash Functions

(a) Merkle-Damgard construction

Used by MD5, SHA-1 and SHA-2. The message is padded (including its length) and divided into fixed-size blocks. A **compression function** f processes the blocks one by one, taking the previous chaining value and the current block as input. The result after the last block is the digest.

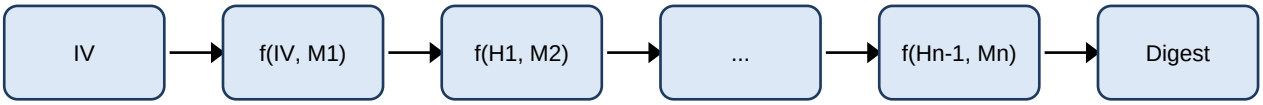

Figure 2: Merkle-Damgard iterative construction

(b) Sponge construction

Used by SHA-3 (Keccak). The input is "absorbed" into a large internal state in blocks, using a fixed permutation, and then the digest is "squeezed" out of the state. This avoids the length-extension weakness of Merkle-Damgard.

(c) Building blocks inside the compression function

- **Bitwise operations:** AND, OR, XOR, NOT and rotations/shifts that mix bits.
- **Modular addition:** addition modulo 2^{32} or 2^{64} adds non-linearity.
- **Substitution and permutation:** rounds of mixing spread the influence of every input bit.
- **Padding:** the message is padded to a multiple of the block size, with the original length appended.
- **Multiple rounds:** e.g. 64 rounds in SHA-256 and 24 rounds in Keccak-f.

1.5 Example of a Simple (Non-Cryptographic) Hash

A simple hash such as H(x) = x mod 10 maps 1234 to 4. It is fixed-size and fast, but it is easy to find collisions (for instance 14 and 24 both give 4) and easy to invert. This is why such functions are unsuitable for security and cryptographic hash functions with strong mathematical and structural design are required.

1.6 Common Cryptographic Hash Functions

Algorithm	Digest size	Status
MD5	128 bits	Broken (collisions found); do not use for security
SHA-1	160 bits	Deprecated (practical collision, 2017)
SHA-256 / SHA-512	256 / 512 bits	Secure, widely used
SHA-3 (Keccak)	224-512 bits	Secure, newest standard

Q2. Discuss the role of cryptographic hash functions in cryptographic applications. Explain their use in data integrity verification, digital signatures, password storage, and message authentication.

Cryptographic hash functions are a core building block of modern security. They give a short, unique "fingerprint" of data, which can be compared, signed, stored or combined with a key. Their main roles are described below.

2.1 Data Integrity Verification

A hash of the original data is calculated and published or stored safely. Later, the receiver recalculates the hash of the received data. If both digests match, the data has not been altered (accidentally or maliciously); if they differ, the data was modified.

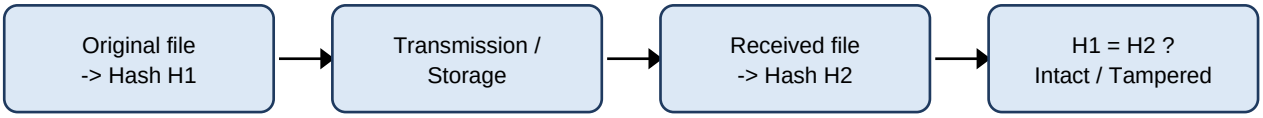

Figure 3: Integrity checking using hashes

Example: Software vendors publish the SHA-256 checksum next to a download. After downloading, the user runs `sha256sum file.iso` and compares the result. Git also identifies every commit by a hash, so history cannot be changed silently.

Limitation: a plain hash detects change but does not prove who sent the data, since an attacker can change both the file and its hash. For that, signatures or MACs are used.

2.2 Digital Signatures

Signing a large message directly with a public-key algorithm (RSA, ECDSA) is slow. Instead, the message is hashed first and only the small digest is signed with the sender's **private key**. The receiver hashes the received message, decrypts (verifies) the signature with the sender's **public key** and compares the two digests.

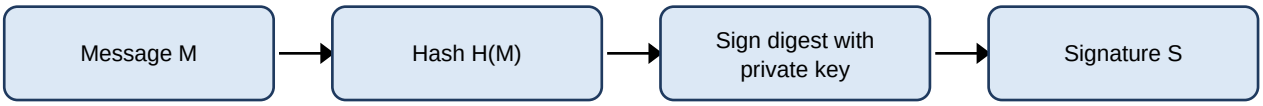

Figure 4: Signature generation (sender)

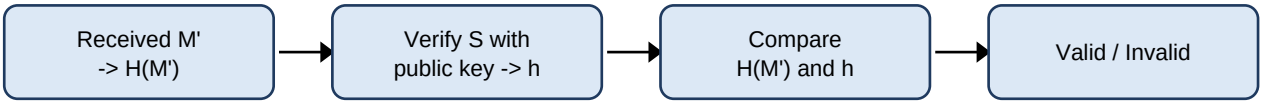

Figure 5: Signature verification (receiver)

The hash gives **efficiency** (a fixed 256-bit value is signed regardless of message size) and **security** (collision resistance ensures an attacker cannot substitute a different message that has the same digest). Digital signatures provide authentication, integrity and non-repudiation.

2.3 Password Storage

Systems should never store passwords in plain text. They store the hash of the password. At login, the entered password is hashed and compared with the stored hash. If the database leaks, attackers get only hashes, not the original passwords.

- **Salting:** a random value (salt) is added to each password before hashing, so identical passwords give different hashes and precomputed rainbow tables become useless. Stored as: salt + H(salt || password).
- **Slow hashing / key stretching:** algorithms such as **bcrypt, scrypt, Argon2 and PBKDF2** repeat the hashing many times to make brute-force attacks expensive. Fast hashes like plain SHA-256 or MD5 are not recommended for passwords.

Example: password "admin123" with salt "X9f2" is stored as SHA-256("X9f2admin123") = d314b8b9efef0671b6dbecf143b4aaaa... (shortened). Another user with the same password but salt "Q7aL" gets a totally different stored value.

2.3.1 Message Authentication

A **Message Authentication Code (MAC)** verifies both integrity and authenticity. The most common hash-based MAC is **HMAC**, which mixes a shared secret key K with the message:

```
HMAC(K, M) = H( (K' xor opad) || H( (K' xor ipad) || M ) )
```

where K' is the key padded to the block size, and ipad and opad are fixed constants (0x36 and 0x5C repeated). The sender transmits M and the HMAC tag. The receiver recomputes the tag with the same key; a match proves that the message came from someone who knows K and was not modified. An attacker without K cannot create a valid tag.

Examples: HMAC-SHA256 is used for API request signing, JSON Web Tokens (HS256), TLS record authentication and IPsec.

2.4 Other Roles

- Key derivation (HKDF, PBKDF2) and pseudo-random number generation.
- Commitment schemes, proof-of-work in blockchains (Bitcoin uses double SHA-256) and Merkle trees.
- Deduplication and quick file identification.

Q3. Explain the advantages and disadvantages of hash cryptography. Discuss the major properties of a secure cryptographic hash function, including preimage resistance, second-preimage resistance, and collision resistance.

3.1 Advantages

- **Speed and efficiency:** digests are computed very quickly even for large data.
- **Fixed-size output:** a compact representation that is easy to store, compare and transmit.
- **No key management:** plain hashing needs no secret keys, which simplifies use.
- **Integrity assurance:** any change in the data is detected through the avalanche effect.
- **One-way protection:** sensitive data such as passwords is not stored in recoverable form.
- **Wide applicability:** signatures, MACs, blockchains, certificates, deduplication and more.
- **Standardised and well studied:** SHA-2 and SHA-3 are published by NIST and widely supported.

3.2 Disadvantages

- **No confidentiality:** hashing does not hide the data; it is not encryption and cannot be decrypted.
- **No authentication on its own:** an attacker can change data and recompute the hash; a key (HMAC) or signature is required.
- **Collisions exist in theory:** the input space is infinite while the output is finite (pigeonhole principle); weak functions (MD5, SHA-1) have practical collisions.
- **Vulnerable to brute force, dictionary and rainbow-table attacks** for weak or unsalted passwords.
- **Length-extension attacks** on Merkle-Damgard hashes (SHA-256 used naively as a MAC).
- **Algorithm ageing:** advances in cryptanalysis and computing power (and eventually quantum computers) reduce security margins, so migration is needed over time.
- **Irreversibility is a drawback** when the original data must be recovered.

3.3 Properties of a Secure Cryptographic Hash Function

(a) Preimage resistance (one-wayness)

Given a digest h, it must be computationally infeasible to find any message M with H(M) = h. For an n-bit hash, brute force needs about $\mathbf{2^n}$ attempts. Example: knowing a stored SHA-256 value should not let an attacker reconstruct the file or password except by guessing.

(b) Second-preimage resistance (weak collision resistance)

Given a message M1, it must be infeasible to find a different message M2 (M2 != M1) such that H(M1) = H(M2). Effort needed is about $\mathbf{2^n}$ operations. Example: an attacker cannot craft a malicious program with the same hash as a given genuine program.

(c) Collision resistance (strong collision resistance)

It must be infeasible to find *any* two different messages M1 and M2 with H(M1) = H(M2). Because of the **birthday paradox**, collisions can be found with only about $\mathbf{2^{n/2}}$ operations, so an n-bit hash gives n/2 bits of collision security (128 bits for SHA-256). Collision resistance implies second-preimage resistance, which is a weaker requirement.

Property	What attacker must find	Brute-force effort (n-bit)	Strength
Preimage resistance	M such that H(M) = h (h given)	2^n	Weakest requirement on attacker
Second-preimage resistance	M2 != M1 with H(M2) = H(M1) (M1 given)	2^n	Stronger than preimage
Collision resistance	Any M1 != M2 with H(M1) = H(M2)	2^(n/2)	Strongest requirement on the function

(d) Other desirable properties

- **Deterministic** and **efficient** to compute.
- **Avalanche effect / pseudo-randomness:** output looks random and each input bit affects about half of the output bits.
- **Fixed output size** for arbitrary input size.
- **Resistance to length-extension** (for newer designs such as SHA-3, SHA-512/256).

Q4. Describe the Secure Hash Algorithm (SHA) family. Explain the development and key features of SHA-1, SHA-2, and SHA-3, highlighting their major differences.

The **Secure Hash Algorithm (SHA)** family is a set of cryptographic hash functions published by the U.S. National Institute of Standards and Technology (NIST) as a Federal Information Processing Standard (FIPS 180 and FIPS 202). They were designed originally with the NSA. SHA-0 (1993) was withdrawn soon after publication because of a flaw; SHA-1, SHA-2 and SHA-3 followed.

4.1 SHA-1

- **Development:** published in 1995 (FIPS 180-1) as a fix of SHA-0; designed by the NSA.
- **Design:** Merkle-Damgard construction; 512-bit blocks; 80 rounds; word size 32 bits; 160-bit digest.
- **Security:** theoretical attacks from 2005; in 2017 Google and CWI announced the first practical collision ("SHAttered"). NIST deprecated SHA-1 and recommended phasing it out by 2030; browsers and certificate authorities already reject SHA-1 certificates.
- **Use today:** only legacy systems and non-security purposes (e.g. older Git object IDs).

4.2 SHA-2

- **Development:** published in 2001 (FIPS 180-2) by the NSA and NIST, with later additions (SHA-224 and SHA-512/t variants).
- **Design:** Merkle-Damgard construction with the Davies-Meyer compression function. SHA-224/256 use 32-bit words, 512-bit blocks and 64 rounds; SHA-384/512 use 64-bit words, 1024-bit blocks and 80 rounds.
- **Members:** SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224 and SHA-512/256.
- **Security:** no practical attacks known; still considered secure. Truncated variants (SHA-384, SHA-512/256) resist length-extension attacks.
- **Use today:** the most widely deployed family: TLS, certificates, Bitcoin, code signing, HMAC.

4.3 SHA-3

- **Development:** NIST ran an open public competition from 2007 after collisions were found in MD5 and SHA-1 and doubts arose about SHA-2. **Keccak** (by Bertoni, Daemen, Peeters and Van Assche) won in 2012 and was standardised as FIPS 202 in 2015.
- **Design: sponge construction** over a 1600-bit state with the Keccak-f permutation (24 rounds). Different internal structure from SHA-2, so a break of SHA-2 would not automatically affect SHA-3.
- **Members:** SHA3-224, SHA3-256, SHA3-384, SHA3-512, and extendable-output functions (XOFs) SHAKE128 and SHAKE256.
- **Features:** immune to length-extension attacks, efficient in hardware, flexible output length.
- **Purpose:** not a replacement for SHA-2 (which is still secure) but a diverse backup standard.

4.4 Comparison of SHA-1, SHA-2 and SHA-3

Feature	SHA-1	SHA-2	SHA-3
Year / standard	1995, FIPS 180-1	2001, FIPS 180-2	2015, FIPS 202
Designer	NSA	NSA / NIST	Keccak team (public competition)
Structure	Merkle-Damgard	Merkle-Damgard (Davies-Meyer)	Sponge construction
Digest size	160 bits	224, 256, 384, 512 bits	224, 256, 384, 512 bits (+ SHAKE variable)
Block / rate	512 bits	512 or 1024 bits	1152-576 bits (depends on variant)
Rounds	80	64 (SHA-256) / 80 (SHA-512)	24
Word size	32 bits	32 / 64 bits	64-bit lanes (1600-bit state)
Length-extension attack	Vulnerable	Vulnerable (except truncated variants)	Not vulnerable
Security status	Broken (practical collisions)	Secure	Secure
Typical use	Legacy only	General purpose, TLS, blockchain	Future-proof / diversity, XOF needs

Major differences in brief: SHA-1 is weak and obsolete; SHA-2 is the strong, current workhorse built on the same design lineage as SHA-1; SHA-3 uses a completely different sponge design, offering structural diversity, resistance to length-extension and flexible output.

Q5. Explain the applications of SHA algorithms in cryptographic protocols and information security. Discuss their role in digital certificates, digital signatures, file integrity checking, and secure communication.

SHA algorithms (mainly SHA-256, SHA-384, SHA-512 and SHA-3) are used in almost every security system because they provide a trusted, fast and tamper-evident fingerprint of data.

5.1 Digital Certificates (X.509 / PKI)

A digital certificate binds a public key to an identity (a website, person or organisation) and is issued by a Certificate Authority (CA). The CA computes the SHA-256 hash of the certificate contents (subject, public key, validity, issuer, etc.) and signs this hash with its private key. The signature algorithm in the certificate is therefore named, for example, *sha256WithRSAEncryption* or *ecdsa-with-SHA384*.

- The browser recomputes the certificate hash and verifies the CA signature using the CA's public key; any change to the certificate invalidates the signature.
- **Certificate fingerprints** (SHA-256 of the certificate) are used to identify and pin certificates.
- SHA-1 certificates were phased out in 2016-2017 because of collision risk; SHA-256 is now the standard.

5.2 Digital Signatures

In RSA-PSS, ECDSA, EdDSA and DSA, the message is first hashed with SHA-2 or SHA-3 and the digest is signed. This gives integrity, authentication and non-repudiation while keeping signing efficient. Examples: code signing for software and drivers, signing of PDF documents, e-mail signing (S/MIME, PGP), Aadhaar eSign and digital signature certificates (DSC) in India, and signing of blockchain transactions.

If the hash were weak (like SHA-1), an attacker could prepare two documents with the same hash, obtain a signature on the harmless one and reuse it on the malicious one. Strong SHA algorithms prevent this.

5.3 File Integrity Checking

- **Checksums for downloads:** Linux ISO files and software packages publish SHA-256 sums (command: `sha256sum filename`).
- **File Integrity Monitoring (FIM):** tools like Tripwire, AIDE and OSSEC store hashes of system files and alert if they change, revealing malware or unauthorised modification.
- **Version control and backups:** Git uses hashes for objects; backup and deduplication systems detect changed or duplicate blocks.
- **Digital forensics:** investigators hash disk images (SHA-256) to prove evidence has not been altered.
- **Malware databases:** antivirus and threat intelligence use SHA-256 hashes as file identifiers (indicators of compromise).

5.4 Secure Communication

- **TLS/SSL (HTTPS):** SHA-256/384 are used in the handshake (Finished message and PRF/HKDF key derivation), in certificate signatures and in HMAC-SHA2 or AEAD constructions (e.g. AES-GCM with SHA-384 in the cipher suite name).
- **IPsec and VPNs:** HMAC-SHA-256 authenticates packets (AH/ESP).
- **SSH:** HMAC-SHA2 for message authentication and host-key fingerprints shown as SHA256:...
- **Secure e-mail:** PGP and S/MIME hash then sign messages.
- **Messaging apps and APIs:** HMAC-SHA256 for webhooks, JWT tokens and API authentication; HKDF-SHA256 for key derivation (e.g. Signal protocol).
- **Wi-Fi security (WPA2/WPA3):** PBKDF2/HMAC-SHA1 or SHA-256 derives keys from the passphrase.

5.5 Other Important Applications

- **Blockchain and cryptocurrency:** Bitcoin uses double SHA-256 for mining (proof of work), block linking and addresses; Merkle trees use SHA-256 to summarise transactions.
- **Password storage:** PBKDF2-HMAC-SHA256 and similar schemes.
- **Random number generation and key derivation:** HMAC-DRBG, HKDF and Hash-DRBG.
- **Timestamping and notarisation:** a hash of a document is timestamped to prove its existence at a given time.

5.6 Summary

Application	Role of SHA	Example
Digital certificates	Hash of the certificate is signed by the CA	X.509 certificate with SHA-256 + RSA
Digital signatures	Message digest signed with private key	Code signing, eSign, PGP
File integrity	Compare checksum before and after	sha256sum of ISO, Tripwire
Secure communication	Handshake, HMAC, key derivation	TLS 1.3, IPsec, SSH

Conclusion: Hash cryptography, and particularly the SHA family, underpins data integrity, authentication, digital trust and secure communication. SHA-1 should be avoided, while SHA-2 and SHA-3 provide strong, standardised protection for modern information systems.